

Monitoramento de Tendências em Manchetes de Notícias

Bernardo Henrique Lebron Machado
Eng. Computação / CEFET-MG
bernardolmachado@gmail.com

Mariana Kaori Yano
Eng. Computação / CEFET-MG
marianakano@gmail.com

Pedro Henrique de Araújo Carvalho
Eng. Computação / CEFET-MG
ypedroaraujo@gmail.com

Resumo—Este trabalho apresenta o desenvolvimento de um sistema computacional para monitoramento de tendências em manchetes de notícias, utilizando como base o conjunto de dados *A Million News Headlines*. O sistema realiza o processamento e a normalização de aproximadamente um milhão de manchetes, identificando as palavras mais frequentes e os termos emergentes ao longo do tempo, além de permitir a busca por manchetes textualmente similares a um conjunto de títulos fornecido pelo usuário. Para tal, foram empregadas estruturas de dados eficientes, como tabelas hash, índice invertido e heap mínimo de capacidade fixa, combinadas ao cálculo do Índice de Jaccard para mensuração de similaridade textual e à taxa de crescimento temporal para detecção de tendências. A solução foi implementada em C++ com programação orientada a objetos, processando os dados de forma incremental sem armazenar o texto bruto em memória. Os resultados demonstram que a combinação dessas estruturas permite processar o corpus completo com eficiência, evidenciando a viabilidade de análises textuais em larga escala por meio de algoritmos clássicos de estruturas de dados.

Index Terms—tabela hash; heap mínimo; índice invertido; similaridade de Jaccard; análise de tendências; processamento de texto

I. INTRODUÇÃO

Atualmente, o avanço tecnológico impulsionou um crescimento em massa na geração de dados. O campo jornalístico é um reflexo nítido desse fenômeno, no qual a velocidade e a quantidade de notícias geradas por minuto inviabilizam qualquer tentativa de análise manual. Tendo em vista esse cenário, e a importância da análise de padrões, comportamentos e tendências, faz-se indispensável o desenvolvimento de um sistema capaz de processar, filtrar e interpretar fluxos textuais de maneira automatizada.

Neste contexto, este trabalho apresenta o desenvolvimento de uma solução computacional para o problema descrito, um sistema voltado à análise de tendências em manchetes de notícias. Embora o conjunto de dados *A Million News Headlines* [4] tenha sido utilizado como a base para o desenvolvimento e validação do projeto, a solução foi modelada de forma genérica, permitindo a análise de fluxos textuais de qualquer base de dados semelhante. O projeto tem como objeto o processamento e tratamento de textos, buscando rastrear palavras relevantes, observar o crescimento de termos emergentes e auxiliar na busca por manchetes similares à manchete alvo.

Para tal, o sistema desenvolvido aplica três principais etapas. Primeiramente, ocorre o tratamento, a padronização

e a filtragem dos dados brutos para eliminação de ruídos. Sequencialmente, realiza-se a análise e a ordenação dos termos para a criação do ranking de palavras recorrentes. Por fim, por meio de métodos de comparação, há a identificação de manchetes similares à notícia alvo.

Portanto, o projeto demonstra a aplicação prática de conceitos de computação para um problema real na área de jornalismo. A união entre o uso de estruturas de dados otimizadas e o processamento automático de textos resulta em uma ferramenta eficaz para a extração de informações relevantes do conjunto de dados, calculando tendências e padrões em larga escala.

II. FUNDAMENTAÇÃO TEÓRICA

Nessa seção, serão apresentados os conceitos teóricos e as estruturas de dados fundamentais para a compreensão do funcionamento do sistema desenvolvido nesse projeto. Para o processo e a análise do volume de dados textuais de forma eficiente, o trabalho utiliza métricas estatísticas e algoritmos da ciência da computação.

A. Índice de Jaccard

O índice de Jaccard é uma métrica utilizada para quantificar a similaridade entre dois conjuntos. Ele é definido pela razão entre o tamanho da interseção e o tamanho da união desses conjuntos, demonstrado por:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (1)$$

O valor desse coeficiente varia no intervalo de 0 a 1, em que o valor 0 indica que os conjuntos não possuem elementos em comum, enquanto o valor 1 afirma que eles possuem uma similaridade máxima.

B. Janelas temporais e taxa de crescimento

A divisão temporal é uma técnica utilizada para analisar como os dados se comportam e se alteram ao longo do tempo. Esse método consiste na segmentação do conjunto de dados em partes menores, chamadas de janelas temporais, o que permite comparar a frequência de aparecimento dos termos ao longo dos períodos.

A partir da contagem obtida em cada janela, calcula-se a taxa de crescimento $C(p)$ de um termo p , comparando a

frequência do termo na primeira janela (F_{J1}) com a frequência na última janela (F_{J5}), expressada na equação:

$$C(p) = \frac{F_{J5}(p) - F_{J1}(p)}{F_{J1}(p) + 1} \quad (2)$$

Ademais, a soma do valor 1 no denominador é utilizada para evitar divisões por zero, o que ocorre quando a palavra analisada não está presente na primeira janela. Como resultado, um valor elevado de $C(p)$ indica um maior crescimento na frequência do termo, ou seja, ele ganhou relevância com o passar do tempo.

C. Tabela Hash

A tabela hash é uma estrutura de dados utilizada para armazenar informações associando chaves a valores, permitindo as operações de busca, inserção e remoção em tempo médio constante e eficiente, independentemente da quantidade de dados armazenados [2].

Para isso, a estrutura utiliza uma função hash, que por sua vez é responsável pela transformação de uma chave de entrada em um valor numérico que representa a posição onde o dado será armazenado na tabela.

Por exemplo, para armazenar a frequência das palavras em um texto, a tabela hash recebe uma palavra como chave e retorna sua contagem como valor:

- Chave: “tecnologia” → Valor: 92
- Chave: “globalização” → Valor: 11
- Chave: “economia” → Valor: 28

Assim, a função hash permite localizar e acessar os elementos rapidamente, reduzindo o custo das operações, por não precisar percorrer todos os elementos armazenados.

D. Índice Invertido

O índice invertido é uma estrutura que inverte a relação natural entre documentos e palavras [3]. Ao contrário de buscas convencionais, que armazenam quais palavras cada documento possui, o índice invertido armazena, para cada palavra única, uma lista de documentos em que ela aparece.

Por exemplo, considerando um conjunto de documentos formado por:

- Documento 1: “Brasil vence jogo decisivo”
- Documento 2: “Jogadores comemoram vitória em um jogo difícil”
- Documento 3: “Brasil vence campeonato”

O índice invertido resultante para algumas das palavras seria:

- “brasil” → [1, 3]
- “vence” → [1, 3]
- “jogo” → [1, 2]
- “difícil” → [2]
- “decisivo” → [1]

Assim, a estrutura permite localizar de forma eficiente os documentos que contêm uma determinada palavra alvo, sem necessidade de percorrer todo o banco de dados. Isso a torna útil em cenários que envolvam a busca de informações dentro de volumes massivos de dados.

E. Heap

A estrutura heap é baseada em árvores binárias que seguem uma propriedade de ordenação entre os seus elementos [2]. Nessa estrutura, cada nó pai mantém uma relação de prioridade com seus filhos, assim permitindo o acesso rápido ao maior ou menor elemento de um grupo de dados. Por conta disso, os heaps são majoritariamente utilizados em algoritmos de ordenação e filas de prioridade.

Existem duas principais variantes de heap: o heap máximo e o heap mínimo. Enquanto no heap máximo o maior elemento está na raiz da árvore — ou seja, cada nó pai possui um valor maior ou igual aos seus filhos — no heap mínimo ocorre o contrário: a raiz contém o menor elemento, portanto, o nó pai possui o valor menor ou igual aos seus filhos.

O heap mínimo é mais utilizado em problemas onde é necessária a conservação dos K maiores elementos de um conjunto sem precisar, necessariamente, ordenar o conjunto. Sua raiz sempre terá o menor dos K melhores elementos vistos até o momento, funcionando como um “lanterna”. Assim, quando um novo elemento chega para comparação, ele é verificado apenas com a raiz: caso seja menor que ela, já é descartado; caso seja maior, o heap se reorganiza para manter sua propriedade.

Para exemplificar, considere o problema de encontrar as 3 palavras mais frequentes em uma lista com 5 palavras:

- “novidade” → 24
- “tecnologia” → 45
- “esporte” → 12
- “educação” → 53
- “governo” → 27

Neste caso o heap mínimo processaria cada palavra uma única vez e manteria apenas as 3 de maior frequência. Portanto, a raiz guardaria o valor 27 (“governo”) como o “lanterna” e os demais (45, 53) estariam armazenados na estrutura heap, sem a necessidade de ordenar a lista toda.

III. METODOLOGIA

Esta seção descreve a modelagem prática e o fluxo lógico seguido para o desenvolvimento do projeto, apresentando como as estruturas de dados expostas anteriormente foram utilizadas para solucionar o problema central: monitoramento de tendências.

A. Visão Geral do Sistema

O sistema é organizado em quatro etapas sequenciais, executadas a partir de dois arquivos de entrada: o corpus de manchetes (`input.csv`) e um arquivo de consultas contendo os títulos a serem pesquisados (`títulos.txt`). A Figura 1 ilustra o fluxo geral de execução.

Na primeira etapa, o módulo `LerArquivo` realiza a leitura e o processamento do corpus, construindo simultaneamente as três estruturas centrais do sistema: o dicionário de frequências (`unordered_map<string, Estatisticas>`), o índice invertido (`unordered_map<string, vector<int>>`) e o vetor de manchetes processadas (`vector<Manchete>`). Essas estruturas são populadas em

uma única passagem sobre o arquivo, após uma pré-leitura para contagem de linhas.

Na segunda etapa, o método `gerarTop100Frequentes()` percorre o dicionário e utiliza uma `MinHeap` de capacidade $K = 100$ para selecionar, em tempo $O(N \log K)$, as palavras com maior frequência global no corpus, onde N representa o total de termos únicos.

Na terceira etapa, o método `gerarTop100Emergentes()` aplica a taxa de crescimento $C(p)$ sobre os dados de frequência por janela já armazenados no dicionário, gerando o ranking de palavras emergentes por meio da mesma estrutura de `MinHeap`. O resultado é preservado em memória para ser reutilizado na etapa seguinte.

Na quarta e última etapa, o método `encontrarSimilaresPorTitulos()` processa cada título do arquivo de consultas, aplica o mesmo pré-processamento do corpus, utiliza o índice invertido para localizar manchetes candidatas e calcula o Índice de Jaccard para cada par. As dez manchetes com maior similaridade são selecionadas por uma `MinHeap` de capacidade $K = 10$ e registradas no arquivo de saída `output.txt`, acompanhadas das *keywords* emergentes presentes em cada resultado.

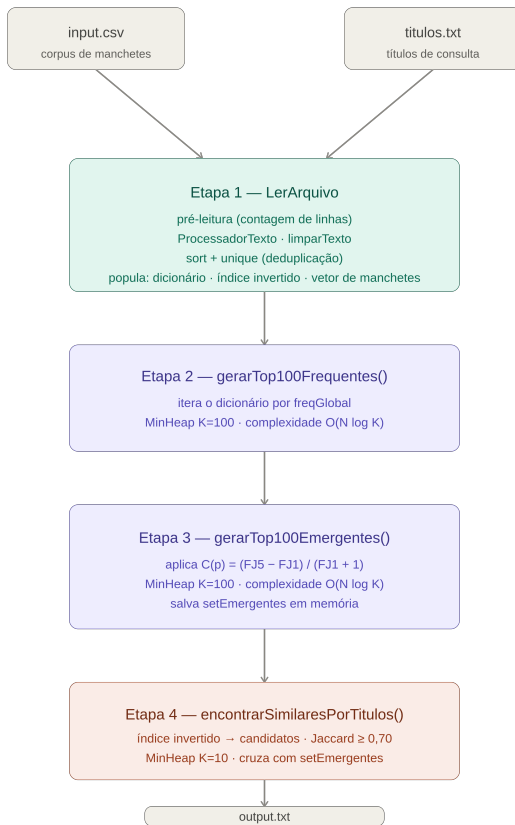


Figura 1. Fluxo geral de execução do sistema.

B. Leitura e Processamento do Arquivo

A leitura do corpus é realizada pelo módulo `LerArquivo`, que executa duas passagens sobre o arquivo `input.csv`. A primeira passagem tem como único objetivo contar o número total de linhas válidas, ou seja, aquelas que contêm ao menos uma vírgula. Esse valor é utilizado para calcular dinamicamente o tamanho de cada janela temporal, dado por $\lfloor N_{total}/5 \rfloor$, onde N_{total} representa o total de manchetes do corpus. Essa abordagem elimina a necessidade de definir um tamanho fixo de janela, tornando o sistema adaptável a bases de dados de diferentes tamanhos.

Para minimizar o número de chamadas ao sistema operacional durante a leitura, é alocado um buffer de 8 MB vinculado ao fluxo de entrada. Esse buffer reduz a latência de I/O ao permitir que grandes blocos de dados sejam carregados de uma só vez na memória.

A segunda passagem realiza o processamento efetivo das manchetes. Para cada linha lida, a posição da vírgula é localizada e o texto à sua direita — correspondente ao título — é encaminhado ao módulo `ProcessadorTexto`. O método `limparTexto()` percorre os caracteres do título aplicando três operações sequenciais: verificação alfabética com `isalpha()`, conversão para minúsculo com `tolower()` e filtragem por comprimento, descartando qualquer token com três ou menos caracteres. O resultado é um vetor de tokens válidos referentes àquela manchete.

Para eliminar palavras repetidas dentro de uma mesma manchete sem o custo de memória adicional de estruturas auxiliares como `unordered_set`, o vetor de tokens é ordenado com `std::sort()` e, em seguida, submetido a `std::unique()` combinado com `erase()`. Essa combinação remove as duplicatas em tempo $O(M \log M)$, onde M é o número de tokens da manchete, operando diretamente sobre o próprio vetor sem alocações extras.

Após a deduplicação, cada token é utilizado para atualizar simultaneamente duas estruturas. No dicionário, os campos `freqGlobal` e `freqJanela[j]` da palavra são incrementados, onde j corresponde à janela calculada por $\lfloor id_{atual}/tamanho_{janela} \rfloor$, limitado ao índice 4. No índice invertido, o identificador da manchete é acrescentado à lista associada à palavra. Por fim, o vetor de tokens é transferido para o vetor de manchetes com `std::move()`, evitando cópias desnecessárias e mantendo o custo de armazenamento proporcional apenas ao número de tokens filtrados, em conformidade com a restrição de não carregar o texto bruto original em memória.

C. Implementação do Heap Mínimo

O heap mínimo é uma das principais estruturas utilizadas no sistema, sendo responsável por manter e atualizar os rankings gerados durante as diferentes etapas de análise. Em vez de utilizar estruturas prontas disponibilizadas pela biblioteca padrão do C++, foi implementada uma versão própria da estrutura, denominada `MinHeap`, desenvolvida especificamente para atender às necessidades de seleção e armazenamento eficiente dos melhores resultados encontrados.

1) *Visão geral da implementação:* A classe `MinHeap` é utilizada em diferentes partes do sistema, sendo reaproveitada em três situações principais: na construção do ranking de palavras mais frequentes, na identificação de palavras emergentes e na seleção das manchetes com maior similaridade textual.

Apesar de serem aplicadas em contextos diferentes, todas as utilizações seguem o mesmo princípio: manter apenas os elementos mais relevantes de acordo com um determinado critério de pontuação. A quantidade máxima de elementos armazenados é definida pelo parâmetro K , informado no momento da criação da estrutura. Dessa forma, são utilizados valores como $K = 100$ para rankings de palavras e $K = 10$ para a seleção das manchetes mais similares.

2) *Estrutura interna:* Cada elemento armazenado pelo heap é representado pela estrutura `HeapNode`, responsável por agrupar as informações necessárias para a comparação. Essa estrutura possui dois campos principais: **palavra**, que armazena a descrição textual do elemento ou o identificador da manchete, e **pontuação**, que representa o valor utilizado como critério de ordenação.

Internamente, a árvore binária do heap é armazenada em um vetor dinâmico (`vector<HeapNode>`). Essa escolha simplifica a implementação, pois elimina a necessidade de utilização de ponteiros e permite que a relação entre os nós seja calculada diretamente a partir dos índices do vetor:

- **Índice do nó pai:** $\lfloor (i - 1) / 2 \rfloor$
- **Índice do filho esquerdo:** $2i + 1$
- **Índice do filho direito:** $2i + 2$

Com essa representação, a navegação entre os elementos da árvore é realizada por operações matemáticas simples, mantendo a estrutura eficiente e compacta.

3) *Inserção e manutenção da propriedade do heap:* A principal característica do heap mínimo é garantir que o menor elemento esteja sempre localizado na raiz da árvore. Para manter essa propriedade após alterações na estrutura, são utilizadas duas operações de reorganização:

- **siftUp:** realizada após uma inserção. O novo elemento é inicialmente colocado no final do vetor e, caso possua uma pontuação menor que a de seu pai, ele é movido para cima na árvore até que a propriedade do heap seja restaurada.
- **heapify:** utilizada quando o elemento da raiz é alterado ou substituído. Nesse caso, o elemento é movimentado para baixo, sendo comparado com seus dois filhos para identificar o menor entre eles e, caso esse filho seja menor que o nó atual, a troca é realizada. Esse processo se repete até que o nó esteja em uma posição onde ambos os filhos possuam pontuação maior ou igual à sua.

4) *Seleção dos K melhores elementos:* O método `insert()` é responsável por controlar a lógica de inserção e filtragem dos elementos. Enquanto a quantidade de elementos armazenados ainda não atingiu o limite máximo K , os novos valores são inseridos normalmente e reorganizados utilizando o método `siftUp`, com custo de execução $O(\log K)$.

Após atingir a capacidade máxima, a raiz do heap passa a representar o menor valor entre os elementos atualmente

armazenados. Dessa forma, ela funciona como um limite de comparação para novos candidatos.

Quando um novo elemento é analisado, sua pontuação é comparada com a raiz da estrutura em tempo constante $O(1)$. Caso o valor seja menor ou igual ao armazenado na raiz, o elemento é descartado, pois não possui pontuação suficiente para entrar no ranking. Caso seja maior, ele substitui a raiz e o processo de `heapify()` é executado para reorganizar o heap em $O(\log K)$.

Esse mecanismo permite manter apenas os K melhores resultados encontrados durante o processamento, evitando a necessidade de armazenar e ordenar todo o conjunto de dados.

5) *Recuperação ordenada com `getSorted()`:* Ao final do processamento, os elementos armazenados precisam ser apresentados em ordem decrescente de pontuação. Como a organização natural do heap mínimo mantém apenas o menor elemento na raiz, a estrutura não fornece diretamente essa ordenação.

Para resolver esse problema, o método `getSorted()` realiza uma cópia dos elementos presentes no vetor interno do heap e aplica uma ordenação sobre essa cópia. Dessa forma, o resultado final é retornado em ordem decrescente sem modificar a estrutura original.

Essa abordagem preserva o estado interno da `MinHeap`, permitindo que ela continue sendo utilizada posteriormente para novas inserções ou análises.

D. Construção das Estruturas de Dados

A construção das estruturas de dados ocorre de forma integrada à leitura do corpus, sendo realizada durante a segunda passagem sobre o arquivo. Essa decisão de projeto elimina a necessidade de etapas separadas de indexação, garantindo que o dicionário, o índice invertido e o vetor de manchetes sejam populados simultaneamente em uma única iteração sobre os dados.

A estrutura central do sistema é o dicionário, implementado como uma tabela hash `unordered_map<string, Estatisticas>`. A `struct Estatisticas` agrupa dois campos em uma única entrada: `freqGlobal`, responsável pela contagem acumulada da palavra ao longo de todo o corpus, e `freqJanela[5]`, um vetor fixo de cinco posições que registra a frequência da palavra em cada janela temporal. Essa unificação permite que ambas as contagens sejam atualizadas em um único acesso à tabela hash por palavra, reduzindo o número de operações de busca em relação a abordagens que utilizam estruturas separadas para cada tipo de contagem.

Paralelamente, o índice invertido é construído na mesma iteração, associando cada palavra a um vetor de identificadores das manchetes em que ela aparece. Por fim, os tokens filtrados de cada manchete são transferidos para o vetor de manchetes por meio de `std::move()`, evitando cópias desnecessárias e mantendo em memória apenas os tokens processados, em conformidade com a restrição de não armazenar o texto bruto original.

E. Ranking de Palavras Frequentes

A geração do ranking das 100 palavras mais frequentes é realizada pelo método `gerarTop100Frequentes()`, que percorre o dicionário e utiliza uma instância da estrutura `MinHeap` com capacidade $K = 100$ para selecionar os termos de maior frequência global sem a necessidade de ordenar o conjunto completo de palavras únicas.

A escolha pelo heap mínimo de capacidade fixa em detrimento de uma ordenação completa do dicionário é motivada pela escala do problema. Com aproximadamente 200 mil palavras únicas no vocabulário do corpus, uma ordenação total apresentaria complexidade $O(N \log N)$. Com a `MinHeap`, o custo é reduzido para $O(N \log K)$, onde $K = 100$ é uma constante pequena, tornando o processo significativamente mais eficiente.

Durante o processamento, cada palavra e sua frequência global são submetidas ao método `insert()` da `MinHeap`. Enquanto a estrutura não atingiu sua capacidade máxima, os elementos são inseridos normalmente. Após atingir $K = 100$ elementos, a raiz do heap passa a representar o menor valor entre os 100 melhores encontrados até o momento. Novos candidatos com frequência inferior à raiz são descartados em tempo $O(1)$, sem qualquer operação adicional. Candidatos superiores substituem a raiz e reorganizam a estrutura via `heapify()` em $O(\log K)$. Ao final da iteração, o método `getSorted()` retorna os 100 elementos em ordem decrescente de frequência, compondo o ranking final.

F. Identificação de Palavras Emergentes

Para descobrir a evolução dos termos ao longo do tempo e isolar aqueles que se tornaram tendências, o sistema implementa uma função com duas fases: mapeamento temporal e filtragem por ordenação parcial.

1) Mapeamento e aplicação da taxa de crescimento:

Nesta etapa, o sistema realiza a aplicação prática da divisão temporal descrita na fundamentação teórica. O conjunto de manchetes é segmentado em cinco blocos equivalentes durante a leitura do arquivo, em que cada manchete é atribuída a uma janela com base na divisão do seu índice sequencial pelo tamanho de cada bloco. As frequências de cada termo são acumuladas separadamente para cada janela dentro do dicionário, uma tabela hash cujos valores armazenam, além da frequência global, um vetor interno de frequências por janela (`freqJanela[5]`).

Para o cálculo da taxa de crescimento, o método `gerarTop100Emergentes()` extrai os valores extremos do vetor, a posição inicial `freqJanela[0]` (F_{J_1}) e a posição final `freqJanela[4]` (F_{J_5}), para utilização na Equação 2. O coeficiente $C(p)$ é calculado em tempo constante $O(1)$ para cada termo e, após isso, é associado à respectiva palavra antes de ser submetido ao processo de ordenação.

2) *Filtragem e ordenação parcial*: Uma abordagem alternativa para a seleção dos termos mais emergentes seria inserir todas as palavras presentes no dicionário em um vetor e realizar uma ordenação completa dos elementos utilizando algoritmos como o Quicksort. Essa estratégia apresenta uma

complexidade de tempo de $O(N \log N)$, considerando N como a quantidade total de palavras únicas analisadas.

Entretanto, considerando a necessidade de processar grandes volumes de dados de maneira eficiente, o sistema utiliza uma estrutura de Min-Heap com tamanho limitado a $K = 100$ elementos. Dessa forma, ao invés de ordenar todo o conjunto de palavras, a estrutura mantém apenas os 100 termos com maiores valores de crescimento ao longo do processamento.

A inserção dos elementos na Min-Heap ocorre de maneira seletiva. Cada palavra, juntamente com seu respectivo coeficiente $C(p)$, é avaliada antes de ser adicionada à estrutura. Enquanto a quantidade de elementos armazenados for menor que 100, a inserção é realizada normalmente, com custo de $O(\log K)$ devido à reorganização do heap.

Após atingir sua capacidade máxima, a raiz da Min-Heap passa a representar o menor coeficiente entre os 100 maiores valores encontrados até aquele momento. Assim, ela funciona como um limite de comparação para os novos elementos analisados. Para cada nova palavra processada, o coeficiente atual é comparado com o valor armazenado na raiz em tempo constante $O(1)$. Caso o novo valor seja menor ou igual ao da raiz, o elemento é descartado, pois não possui relevância suficiente para entrar no ranking. Caso apresente um valor maior, o elemento da raiz é removido e substituído pelo novo termo, sendo necessário executar o processo de reorganização do heap, com custo de $O(\log K)$.

Ao final da análise do dicionário, a estrutura garante que apenas as 100 palavras com maior crescimento sejam mantidas. Em seguida, o método `getSorted()` é utilizado para extrair os elementos armazenados e reorganizá-los em ordem decrescente de acordo com o índice de emergência, permitindo a geração do ranking final.

G. Análise de Similaridade

A análise de similaridade representa a etapa final do sistema, sendo responsável por integrar as estruturas de dados construídas durante as fases anteriores para realizar consultas textuais fornecidas pelo usuário. A entrada desse módulo é o arquivo `titulos.txt`, no qual cada linha representa um título que será utilizado como consulta dentro da base de notícias.

1) *Leitura e processamento da entrada*: O sistema realiza a leitura do arquivo de consultas de maneira sequencial, processando cada linha individualmente. Cada título é tratado como uma consulta independente e passa pelo mesmo processo de pré-processamento aplicado às manchetes durante a construção inicial da base de dados.

Esse processamento envolve etapas como normalização dos caracteres, tokenização, remoção de termos irrelevantes e eliminação de palavras repetidas dentro da mesma consulta. Essa padronização garante que tanto os títulos pesquisados quanto as notícias armazenadas sejam representados da mesma forma, permitindo que a comparação textual seja realizada de maneira consistente.

2) *Aplicação integrada das estruturas*: Após o processamento da consulta, o sistema utiliza as estruturas de indexação e ordenação desenvolvidas para localizar e classificar

as notícias mais semelhantes. Para cada título analisado, são realizadas as seguintes etapas:

- O Índice Invertido é utilizado para encontrar rapidamente as manchetes que possuem termos em comum com a consulta, evitando a necessidade de percorrer todo o conjunto de notícias armazenadas.
- Para cada manchete candidata encontrada, é calculado o Índice de Jaccard, responsável por medir o grau de similaridade entre os conjuntos de tokens da consulta e da notícia. Apenas manchetes com $J(A, B) \geq 0,70$ são consideradas similares.
- Os resultados são inseridos em uma Min-Heap com capacidade máxima de 10 elementos. Essa estrutura permite manter apenas as manchetes mais relevantes durante o processamento, evitando a necessidade de ordenar todos os resultados encontrados. Ao final, o método `getSorted()` é aplicado sobre o heap para ordenar os 10 elementos selecionados em ordem decrescente de similaridade.
- Após a seleção das melhores correspondências, é realizada uma consulta na tabela hash `setEmergentes` para verificar a presença de palavras associadas às tendências emergentes. Essa busca é realizada em tempo médio constante $O(1)$, permitindo identificar rapidamente quais keywords emergentes aparecem nas manchetes selecionadas.

3) *Geração do relatório de saída*: Após o processamento de cada consulta, os resultados são organizados e gravados no arquivo de saída `output.txt`. O relatório gerado apresenta, para cada título pesquisado:

- Os tokens processados e utilizados na comparação;
- As manchetes encontradas, ordenadas de acordo com o maior valor de similaridade calculado pelo Índice de Jaccard;
- Os identificadores (IDs) e os respectivos textos reconstruídos das notícias selecionadas;
- As keywords emergentes identificadas em cada manchete retornada.

Caso nenhuma notícia alcance o valor mínimo de similaridade estabelecido para determinada consulta, o sistema registra essa ausência de resultados no relatório final. Dessa forma, a saída mantém uma estrutura organizada e representa corretamente todas as consultas realizadas, incluindo aquelas sem correspondências encontradas.

IV. RESULTADOS

A. Ambiente de Desenvolvimento

Para avaliar o desempenho do sistema, foram realizadas cinco execuções consecutivas sobre o corpus completo de manchetes, com o mesmo arquivo de consultas, em dois ambientes distintos: Windows 11 e Zorin OS (Linux). Os testes foram conduzidos no mesmo equipamento em ambos os casos, cujas especificações são apresentadas na Tabela I.

Tabela I
ESPECIFICAÇÕES DO AMBIENTE DE TESTE

Componente	Especificação
Processador	Intel Core Ultra 7 155H @ 4,80 GHz
Memória RAM	16 GB
Armazenamento	SSD 512 GB
GPU	Intel Arc (integrada)
Modelo	Samsung Galaxy Book4 Pro
S.O. (Linux)	Zorin OS / GCC 13.3.0
S.O. (Windows)	Windows 11 / MSVC

B. Bibliotecas utilizadas

O sistema foi desenvolvido utilizando bibliotecas pertencentes a biblioteca padrão do C++. As bibliotecas utilizadas são:

1) `<chrono>`: Os tempos de execução foram coletados por meio da biblioteca `<chrono>`, utilizando `high_resolution_clock` com granularidade em milissegundos, evitando assim qualquer dependência de funções específicas de plataforma.

2) `<iostream>`: Utilizada para operações de entrada e saída padrão, sendo responsável pela exibição das mensagens de progresso no terminal durante a execução.

3) `<vector>`: Utilizada para armazenamento de coleções dinâmicas de elementos, como o vetor de manchetes, o de tokens de cada manchete e o vetor interno do MinHeap.

4) `<string>`: Utilizada para manipulação de cadeias de caracteres, representando os tokens extraídos e as chaves dos dicionários.

5) `unordered_map`: Utilizada para implementar o dicionário principal (`dicionario`), que, por sua vez, associa cada palavra às suas estatísticas de frequência global por janela e o índice invertido (`indiceInvertido`), que associa cada palavra à lista de manchetes onde ela aparece.

6) `<unordered_set>`: Utilizada para armazenar o conjunto de palavras emergentes (`setEmergentes`), permitindo verificações de pertencimento em tempo médio constante $O(1)$ durante a etapa de análise de similaridade.

7) `<algorithm>`: Utilizada na deduplicação dos tokens via `sort` e `unique` durante a leitura do arquivo, e na ordenação dos elementos retornados pelo método `getSorted()` da classe MinHeap.

8) `<iomanip>`: Utilizada para a formatação da saída dos relatórios, controlando o alinhamento das colunas e a precisão dos valores de ponto flutuante exibidos.

9) `<sstream>`: Utilizada para criar o buffer `lixreira`, um fluxo de saída em memória utilizado para descartar a saída das etapas internas que não devem ser exibidas no terminal nem gravadas no arquivo de saída.

10) `<fstream>`: Utilizada para leitura do arquivo de dados (`input.csv`) e do arquivo de títulos (`titulos.txt`), bem como para a criação e escrita do arquivo de saída (`output.txt`).

11) `<cctype>`: Utilizada no `ProcessadorTexto.cpp` pelas funções `isalpha()`, que verifica se um caractere é alfabético, e `tolower()`, que

converte caracteres para minúsculas durante a normalização do texto.

C. Análise dos resultados

A Tabela II apresenta os tempos de execução obtidos nas cinco execuções e as respectivas médias para cada sistema operacional.

Tabela II
TEMPOS DE EXECUÇÃO (s) POR SISTEMA OPERACIONAL

Execução	Windows (s)	Linux (s)
1	6,49	1,80
2	6,41	1,81
3	13,61	1,84
4	6,40	1,76
5	6,37	1,73
Média	7,86	1,79

Os resultados evidenciam uma diferença expressiva de desempenho entre os dois sistemas operacionais, mesmo com o código-fonte e o hardware sendo idênticos. No Linux, os tempos se mantiveram estáveis ao longo das cinco execuções, com variação inferior a 0,12 s, refletindo um comportamento previsível e consistente. No Windows, quatro das cinco execuções se concentraram em torno de 6,40 s, porém a terceira execução registrou 13,61 s — aproximadamente o dobro das demais — caracterizando um *outlier* que eleva a média para 7,86 s. Desconsiderando essa execução atípica, a média das quatro execuções regulares no Windows seria de 6,42 s, ainda 3,6 vezes superior à média obtida no Linux. O pico isolado na terceira execução do Windows é consistente com interferências de processos em segundo plano, como atualizações automáticas do sistema ou varreduras de antivírus, que podem interromper a execução do processo por um intervalo de tempo não determinístico. Essa divergência é esperada e pode ser atribuída a um conjunto de fatores estruturais de cada plataforma.

O primeiro fator diz respeito ao sistema de arquivos. O Windows utiliza NTFS, enquanto o Linux utiliza ext4. As operações de leitura sequencial de arquivos grandes — como o `input.csv` de aproximadamente 1 milhão de linhas — tendem a ser mais eficientes no ext4, cujo modelo de alocação é otimizado para acesso sequencial e apresenta menor *overhead* de metadados em comparação ao NTFS [5].

O segundo fator está relacionado ao gerenciador de memória dinâmica. O sistema utiliza extensivamente `unordered_map`, `vector` e alocações de `string`, todas dependentes do alocador padrão da plataforma. O alocador do ambiente GNU/Linux (`ptmalloc2`, derivado do `glibc`) costuma apresentar menor fragmentação e menor latência em padrões de alocação de muitos objetos pequenos do que o alocador padrão do ambiente Windows (`HeapAlloc`) [6].

O terceiro fator é o escalonador de processos. O escalonador do Linux (*Completely Fair Scheduler* — CFS) tende a conceder fatias de CPU mais longas e previsíveis a processos intensivos em computação, reduzindo a quantidade de trocas

de contexto durante a execução [7]. No Windows, o escalonador prioriza a responsividade do sistema, o que pode introduzir interrupções adicionais em processos de longa duração.

Por fim, o cache de página do sistema operacional influencia diretamente o tempo de leitura do arquivo. No Linux, após a primeira execução, o conteúdo do `input.csv` tende a permanecer no cache de página do kernel, reduzindo o custo de I/O nas execuções subsequentes. Esse comportamento explica, em parte, a variação observada entre a primeira e as demais execuções nos dois ambientes.

V. CONCLUSÃO

Este trabalho apresentou o desenvolvimento de um sistema computacional para monitoramento de tendências em manchetes de notícias, com base em estruturas de dados clássicas implementadas em C++ com programação orientada a objetos. O sistema demonstrou ser capaz de processar um corpus de aproximadamente um milhão de manchetes de forma eficiente, sem armazenar o texto bruto em memória, em conformidade com as restrições impostas pelo problema.

A utilização da tabela hash como estrutura central de contagem, combinada ao índice invertido para a localização eficiente de candidatos à similaridade, permitiu reduzir significativamente o espaço de busca em relação a uma abordagem de comparação exaustiva $O(n^2)$. A adoção do heap mínimo de capacidade fixa K para os rankings de palavras frequentes e emergentes viabilizou a seleção dos melhores resultados em tempo $O(N \log K)$, evitando a ordenação completa de vocabulários com centenas de milhares de entradas. O cálculo do Índice de Jaccard sobre conjuntos deduplicados de tokens, com *threshold* de 70%, mostrou-se uma métrica adequada para capturar a similaridade textual entre manchetes curtas.

A comparação de desempenho entre Windows e Linux evidenciou que, apesar do código-fonte ser idêntico, fatores de plataforma — como sistema de arquivos, alocador de memória, escalonador de processos e política de cache de página — exercem influência mensurável sobre o tempo de execução. Esse resultado reforça a importância de considerar o ambiente de implantação ao analisar o desempenho de sistemas que realizam leitura intensiva de arquivos e alocações frequentes de memória.

Como possíveis trabalhos futuros, destaca-se a substituição do cálculo exato de Jaccard por técnicas de estimativa como MinHash e Locality-Sensitive Hashing (LSH), que permitiriam ampliar a escala do módulo de similaridade para corpus ainda maiores sem degradação significativa de precisão. Outra extensão natural seria a adição de um dicionário de *stopwords* para filtrar termos gramaticalmente irrelevantes, tornando os rankings de palavras frequentes e emergentes mais informativos.

REFERÊNCIAS

- [1] Repositório do projeto no GitHub. Disponível em: <https://github.com/marixzh/MonitoramentoDeTendencias>
- [2] T. H. Cormen, C. E. Leiserson, R. L. Rivest e C. Stein, *Introduction to Algorithms*, 4ª ed. Cambridge, MA: MIT Press, 2022.

- [3] C. D. Manning, P. Raghavan e H. Schütze, *Introduction to Information Retrieval*. Cambridge: Cambridge University Press, 2008. Disponível em: <https://nlp.stanford.edu/IR-book/>
- [4] A. Kulkarni, “A Million News Headlines”, Kaggle, 2017. Disponível em: <https://www.kaggle.com/datasets/therohk/million-headlines>
- [5] D. P. Bovet e M. Cesati, *Understanding the Linux Kernel*, 3ª ed. Sebastopol, CA: O’Reilly Media, 2005.
- [6] S. Ghemawat e P. Menage, “TCMalloc: Thread-Caching Malloc”, Google, 2009. Disponível em: <https://goog-perftools.sourceforge.net/doc/tcmalloc.html>
- [7] R. Love, *Linux Kernel Development*, 3ª ed. Upper Saddle River, NJ: Addison-Wesley, 2010.